

Lecture 2 Polynomial Methods

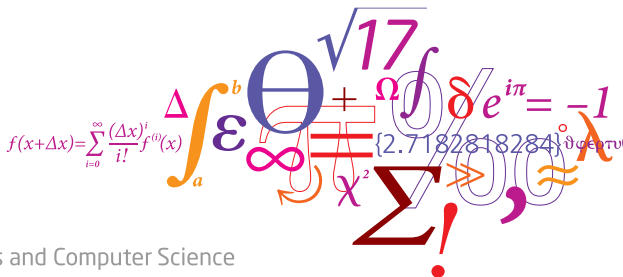
02689 Advanced Numerical Methods For Differential Equations

Allan P. Engsig-Karup

Scientific Computing Section

DTU Compute

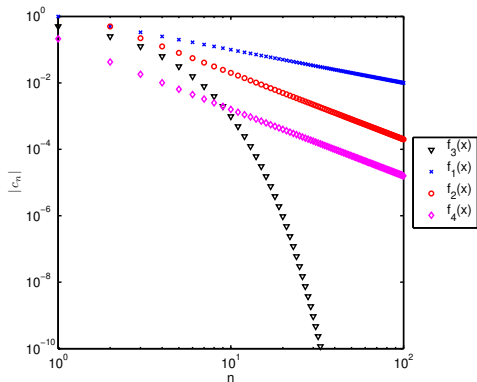
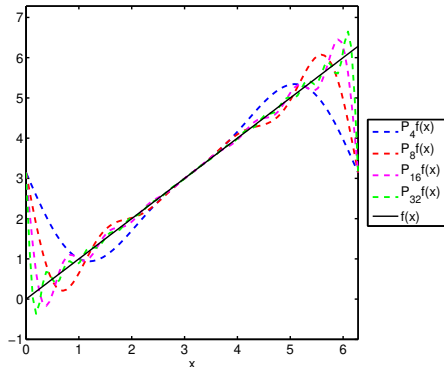
Technical University of Denmark



DTU Compute

Department of Applied Mathematics and Computer Science

Properties of Fourier Methods

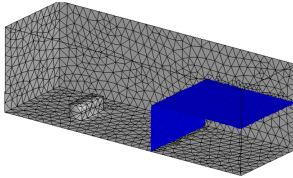


Properties of Fourier Methods

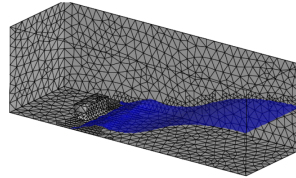
- Spectral convergence if both solution and its derivatives are periodic and smooth ([accuracy](#)) and otherwise (sometimes slow) algebraic convergence similar to finite difference methods.
- Fast $\mathcal{O}(n \log n)$ evaluation of DFT via FFT ([efficiency](#)).

On Real-world Applications

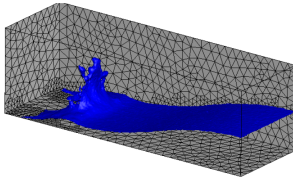
Applications related to the real world may be difficult to describe using periodic assumptions.... what can be done?



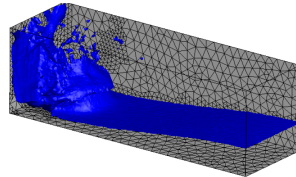
(a) $t = 0.0$



(b) $t = 0.35$



(c) $t = 0.5$



(d) $t = 0.8$

Computational Fluid Dynamics : Incompressible Multiphase Flows using Discontinuous Galerkin Methods. Source: Karakus, Warburton, Aksel & Sert (2016).
compute

From Periodic to Non-periodic Domains

The periodicity requirement in Fourier Methods **destroys** fast convergence for representation of smooth non-periodic functions.

The idea of the Fourier Series representation of the form

$$f(x) = \sum_{n=-\infty}^{\infty} c_n e^{ikx}$$

can be generalized into a representation of the form

$$f(x) = \sum_{n=0}^{\infty} \hat{f}_n \phi_n(x)$$

Thus, to represent a function $f(x)$ we need to

- Choose a set of global basis functions $\{\phi_n(x)\}_{n=0}^{\infty}$.
- Find ways to determine the coefficients $\{\hat{f}_n\}_{n=0}^{\infty}$.

Can we still get fast spectral convergence without periodicity?

Polynomial representation

Consider the following idea

- Use orthogonal polynomials to represent/approximate functions.

Why?

- Polynomials are easy to evaluate, integrate and differentiate.

Important classes of methods in applications that are based on polynomial approximations

- **Finite difference methods & Finite Volume Methods**
(local approximations using polynomials)
- **Spectral methods**
(high-order polynomials on single domain).
- **Finite element methods**
(low-order polynomials on many domains).

Polynomial Methods

Assume a general polynomial series representation of a square-integrable function $f(x)$ on a domain $x \in [a, b]$

$$f(x) = \sum_{n=0}^{\infty} \hat{f}_n \phi_n(x)$$

with coefficients defined from weighted inner products

$$\hat{f}_n = \frac{1}{\gamma_n} \int_a^b f(x) \phi_n(x) w(x) dx$$

where $w(x) > 0$ is a weight function that guarantee orthogonality of the basis functions such that

$$(\phi_m(x), \phi_n(x))_w = \int_a^b \phi_m(x) \phi_n(x) w(x) dx = \delta_{mn} \gamma_n$$

with normalization constant γ_n .

Polynomial Methods

For practical purposes the polynomial series is approximated by a truncated version of the form

$$\mathcal{P}_N f(x) = \sum_{n=0}^N \hat{f}_n \phi_n(x)$$

which represents a polynomial of degree N .

If we introduce τ as the truncation error, then we find

$$f(x) = \mathcal{P}_N f(x) + \tau, \quad \tau = \sum_{n=N+1}^{\infty} \hat{f}_n \phi_n(x)$$

with the error dependent on how rapid $\hat{f}_n$ decays for $n \rightarrow \infty$.

Polynomial Methods

We need ways to

- Construct orthogonal polynomial basis functions efficiently.
- Easily generate and evaluate such polynomial basis functions.

Sturm-Liouville Problem

The Sturm-Liouville Problem is defined as

$$\frac{d}{dx} \left(p(x) \frac{du}{dx} \right) + (q(x) + \lambda w(x))u = 0$$

for suitable boundary conditions on $u(x)$ and $p(x) > 0$, $p'(x)$, $q(x)$ and $w(x) > 0$ continuous functions and λ real-valued in $x \in [a, b]$.

If $p(x) = 0$ for at least one $x \in [a, b]$ the problem is referred to as *singular* and otherwise *regular*.

The Sturm-Liouville problem is important since it can be formulated as an eigenvalue problem

$$\mathcal{D}u_n = \lambda_n u_n, \quad \mathcal{D} = - \left(p(x) \frac{d^2}{dx^2} + \frac{d}{dx} p(x) \frac{d}{dx} + q(x) \right)$$

with eigenfunctions u_n for a complete orthogonal polynomial basis $\{u_n\}_{n=0}^{\infty}$ in $L_w^2(a, b)$.

Jacobi polynomials

The class of Jacobi polynomials $P_n^{(\alpha,\beta)}(x)$ of order n includes all polynomial solutions to the singular Sturm-Liouville problem

$$\begin{aligned}\frac{d}{dx}(1-x^2)w(x)\frac{d}{dx}P_n^{(\alpha,\beta)}(x) + \lambda w(x)P_n^{(\alpha,\beta)}(x) &= 0, \\ \lambda &= n(n + \alpha + \beta + 1), \\ w(x) &= (1-x)^\alpha(1+x)^\beta\end{aligned}$$

for $x \in [-1, 1]$ and $\alpha, \beta > -1$.

A consequence of the Sturm-Liouville problem is the weighted inner product for orthogonal Jacobi polynomials

$$\int_{-1}^1 P_m^{(\alpha,\beta)}(x)P_n^{(\alpha,\beta)}(x)w(x)dx = \gamma_n^{(\alpha,\beta)}\delta_{mn}$$

which can therefore be used for the construction of spectral methods in non-periodic domains.

Continuous expansion based on Jacobi polynomials

Introduce the continuous polynomial expansion of a function

$$u(x) = \sum_{n=0}^{\infty} \hat{u}_n P_n^{(\alpha, \beta)}(x)$$

with expansion coefficients

$$\hat{u}_n = \frac{1}{\gamma_n^{(\alpha, \beta)}} \int_{-1}^1 u(x) P_n^{(\alpha, \beta)}(x) (1-x)^\alpha (1+x)^\beta dx$$

where $w(x) = (1-x)^\alpha (1+x)^\beta$ is a weight function that guarantee orthogonality of the basis functions

$$\int_{-1}^1 P_m^{(\alpha, \beta)}(x) P_n^{(\alpha, \beta)}(x) (1-x)^\alpha (1+x)^\beta dx = \gamma_n^{(\alpha, \beta)} \delta_{mn}$$

with normalization constant for Jacobi polynomials given as

$$\gamma_n^{(\alpha, \beta)} = \left\| P_n^{(\alpha, \beta)} \right\|_{L_w^2[-1, 1]}^2 = 2^{\alpha+\beta+1} \frac{\Gamma(n+\alpha+1)\Gamma(n+\beta+1)}{n!(2n+\alpha+\beta+1)\Gamma(n+\alpha+\beta+1)}$$

with $\Gamma(\cdot)$ the Euler Gamma function.

Generation and evaluation of Jacobi polynomials

Recursion relation for **evaluating** Jacobi polynomials for $x \in [-1, 1]$

$$P_0^{(\alpha, \beta)}(x) = 1$$

$$P_1^{(\alpha, \beta)}(x) = \frac{1}{2} (\alpha - \beta + (\alpha + \beta + 2)x)$$

$$a_{n+1, n}^{(\alpha, \beta)} P_{n+1}^{(\alpha, \beta)}(x) = \left(a_{n, n}^{(\alpha, \beta)} + x \right) P_n^{(\alpha, \beta)}(x) - a_{n-1, n}^{(\alpha, \beta)} P_{n-1}^{(\alpha, \beta)}(x)$$

with coefficients for $n \geq 0$

$$a_{n-1, n}^{(\alpha, \beta)} = \frac{2(n + \alpha)(n + \beta)}{(2n + \alpha + \beta + 1)(2n + \alpha + \beta)}$$

$$a_{n, n}^{(\alpha, \beta)} = \frac{\alpha^2 - \beta^2}{(2n + \alpha + \beta + 2)(2n + \alpha + \beta)}$$

$$a_{n+1, n}^{(\alpha, \beta)} = \frac{2(n + 1)(n + \alpha + \beta + 1)}{(2n + \alpha + \beta + 2)(2n + \alpha + \beta + 1)}$$

and for $n = 0$

$$a_{-1, 0}^{(\alpha, \beta)} = 0$$

Special Jacobi polynomial classes

Some of the most widely used polynomial basis functions belongs to the class of ultraspherical polynomials with $\alpha = \beta$.

Two important polynomial classes are

- Legendre polynomials ($\alpha = \beta = 0$)
 - Corresponds to Jacobi polynomials $P_n^{(0,0)}$
 - $w(x) = 1$

and

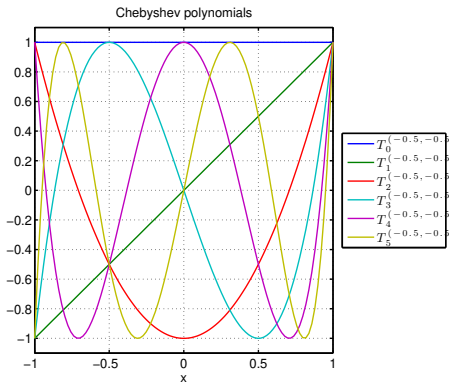
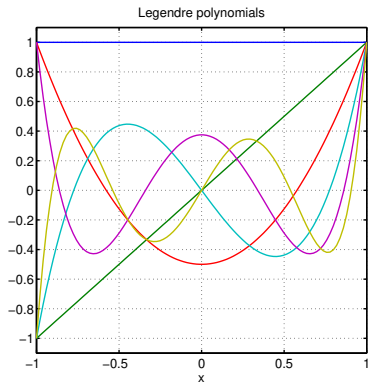
- Chebyshev polynomials ($\alpha = \beta = -\frac{1}{2}$)
 - Corresponds to normalized Jacobi polynomials of the form

$$T_n(x) = \frac{\Gamma(n+1)\Gamma(\frac{1}{2})}{\Gamma(n+\frac{1}{2})} P_n^{(-\frac{1}{2}, -\frac{1}{2})}(x)$$

- $w(x) = \frac{1}{\sqrt{1-x^2}}$

Jacobi polynomials

Illustrations of the first few Legendre and Chebyshev polynomials.



Generation and evaluation of derivatives of Jacobi polynomials

A recursion relation for evaluating the k 'th derivative of Jacobi polynomials in $x \in [-1, 1]$ is

$$\frac{d^k}{dx^k} P_n^{(\alpha, \beta)}(x) = \frac{\Gamma(\alpha + \beta + n + 1 + k)}{2^k \Gamma(\alpha + \beta + n + 1)} P_{n-k}^{(\alpha+k, \beta+k)}(x)$$

Specifically, for the first derivative we find

$$\frac{d}{dx} P_n^{(\alpha, \beta)}(x) = \frac{1}{2}(\alpha + \beta + n + 1) P_{n-1}^{(\alpha+1, \beta+1)}(x)$$

Thus, evaluation can take place using the recurrence for evaluating Jacobi polynomials.

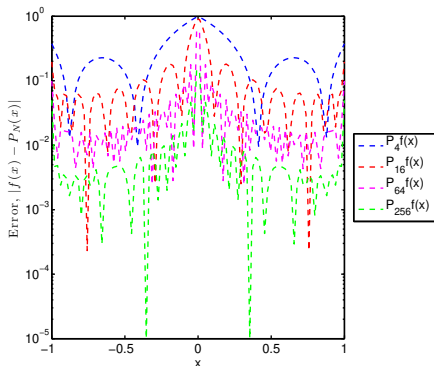
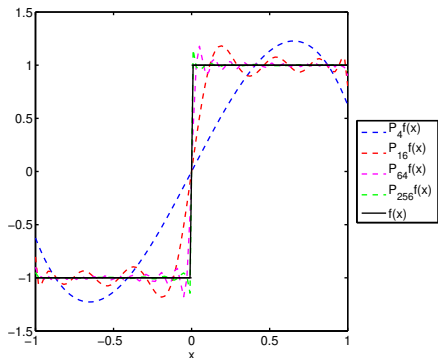
Convergence

Consider the piece-wise continuous function on $x \in [-1, 1]$

$$f_1(x) = \begin{cases} -1 & , x \leq 0 \\ 1 & , x > 0 \end{cases}$$

which can be shown to have the Legendre expansion coefficients

$$\hat{f}_k = \frac{1}{2}(1 + 2k) \left(\frac{\sqrt{\pi}}{\Gamma(1 - \frac{1}{2})\Gamma(\frac{3+k}{2})} \right) - \delta_{k0}$$



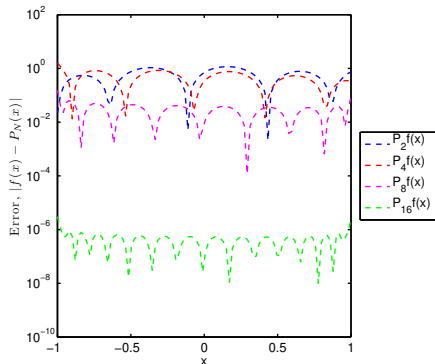
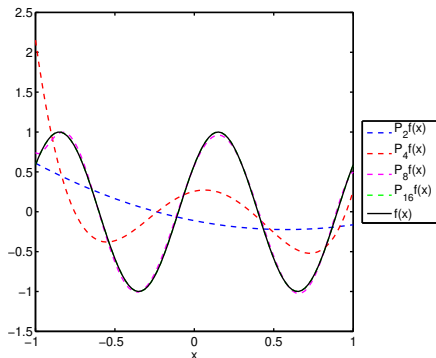
Convergence

Consider the approximation of the $C^\infty([-1, 1])$ function

$$f_3(x) = \sin(2\pi(x + 0.1))$$

which can be shown to have the Legendre expansion coefficients

$$\hat{f}_k = \frac{2k+1}{2} J_{k+1/2}(2\pi) \sin(0.2\pi + k\pi/2)$$



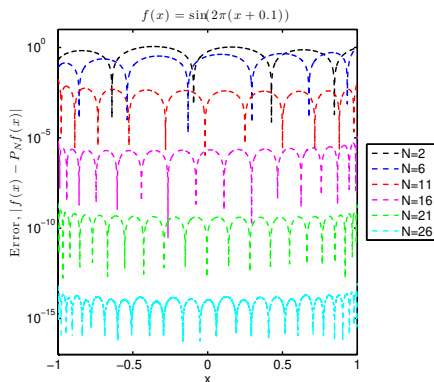
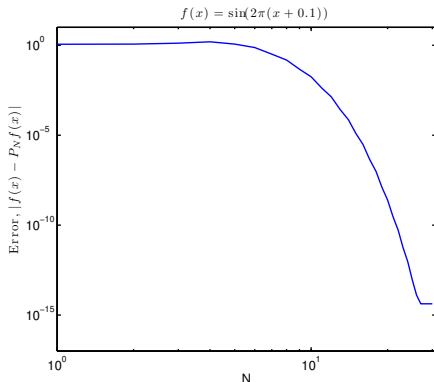
Convergence

Consider the approximation of the $C^\infty([-1, 1])$ function

$$f(x) = \sin(2\pi(x + 0.1))$$

which can be shown to have the Legendre expansion coefficients

$$\hat{f}_k = \frac{2k+1}{2} J_{k+1/2}(2\pi) \sin(0.2\pi + k\pi/2)$$



Orthonormal Jacobi polynomials

For practical computations it can be useful to use the **orthonormal** Jacobi polynomials $\tilde{P}_n^{(\alpha,\beta)}(x)$ which fulfills

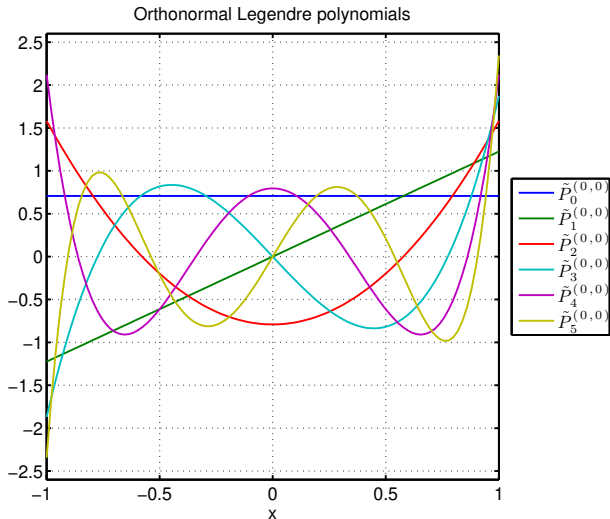
$$\int_{-1}^1 \tilde{P}_m^{(\alpha,\beta)}(x) \tilde{P}_n^{(\alpha,\beta)}(x) w(x) dx = \delta_{mn}$$

and are defined through normalization of Jacobi polynomials such that

$$\tilde{P}_n^{(\alpha,\beta)}(x) = \frac{1}{\sqrt{\gamma_n^{(\alpha,\beta)}}} P_n^{(\alpha,\beta)}(x)$$

Orthonormal Jacobi polynomials

Illustration of the first few orthonormal Legendre polynomials.



Generation and evaluation of derivatives of orthonormal Jacobi polynomials

The following recursion relation is given for evaluating the k 'th derivative of orthonormal Jacobi polynomials in $x \in [-1, 1]$

$$\frac{d^k}{dx^k} \tilde{P}_n^{(\alpha, \beta)}(x) = \frac{\Gamma(\alpha + \beta + n + 1 + k)}{2^k \Gamma(\alpha + \beta + n + 1)} \sqrt{\frac{\gamma_{n-k}^{(\alpha+k, \beta+k)}}{\gamma_n^{(\alpha, \beta)}}} \tilde{P}_{n-k}^{(\alpha+k, \beta+k)}(x)$$

For example, from this relationship we can derive the useful relations

$$\begin{aligned} \frac{d}{dx} \tilde{P}_n^{(\alpha, \beta)}(x) &= \sqrt{n(n + \alpha + \beta + 1)} \tilde{P}_{n-1}^{(\alpha+1, \beta+1)}(x) \\ \frac{d^2}{dx^2} \tilde{P}_n^{(\alpha, \beta)}(x) &= 2 \sqrt{\frac{n(n-1)(\alpha + \beta + n + 1)}{(\alpha + \beta + n + 2)}} \tilde{P}_{n-2}^{(\alpha+2, \beta+2)}(x) \end{aligned}$$

Convergence of polynomial series

The Weierstrass Theorem states that

$$\lim_{N \rightarrow \infty} \|u(x) - \mathcal{P}_N u(x)\|_w = \|\tau\|_w = 0$$

for truncated Polynomial series constructed from a system of algebraic polynomials $\{\phi_n\}_{n=0}^{\infty}$ that are mutually orthogonal over the interval $x \in [-1, 1]$ with respect to a weight function, i.e.

$$\int_{-1}^1 \phi_m(x) \phi_n(x) w(x) dx = \delta_{mn} \gamma_n$$

The norm of the truncation error τ is

$$\|\tau\|_{L_w^2}^2 = \sum_{k=N+1}^{\infty} |\hat{f}_k|^2 \|\phi_k\|_{L_w^2}^2$$

which makes it clear that the rate of convergence depends on the decay of the expansion coefficients.

Convergence of polynomial series

But how fast does the series converge?

Suppose $f(x)$ has continuous derivatives of orders $p = 0, 1, \dots, m - 1$ and $f^{(m)}(x)$ is integrable.

For ultraspherical polynomials $P_n^{(\alpha, \alpha)}$ the following result exist¹

$$\|f(x) - \mathcal{P}_N f(x)\|_{L_w^2(-1,1)} \leq CN^{-p} \|f(x)\|_{H_w^p(-1,1)}$$

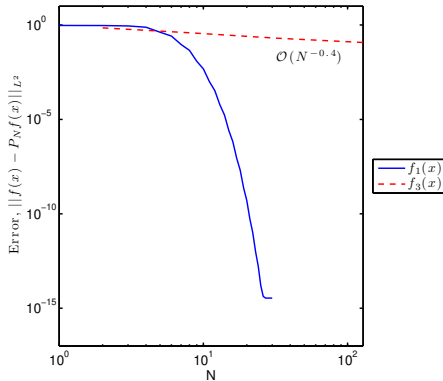
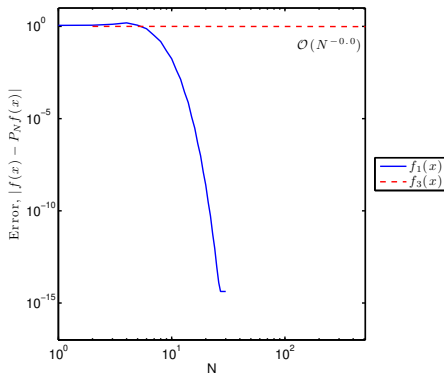
which provides an upper bound for the truncation error. if $f(x)$ only have p classical continuous derivatives in $[-1, 1]$ which are integrable. In which case convergence is algebraic

$$\tau \sim \frac{1}{N^p}, \quad N \rightarrow \infty$$

This implies that if $f(x)$ is smooth ($p = \infty$ or very large), the Polynomial series converges rapidly and uniformly.

¹Technical proof detailed in Hesthaven, Gottlieb & Gottlieb (2007).

Convergence rate of polynomial series and norms



- Algebraic convergence if $f(x) \in C^p(a, b)$ for $n \rightarrow \infty$ and p finite.
- Spectral convergence if $f(x) \in C^\infty(a, b)$ for $n \rightarrow \infty$.
- If coefficients decay slowly, it requires a lot of work to evaluate $f(x)$ to high accuracy (may not be practical)
- Decay of errors depends on norm used (e.g. point-wise or mean errors).

Polynomial series in practice

$$f(x) = \sum_{n=0}^{\infty} \hat{f}_n \phi_n(x), \quad \hat{f}_n = \frac{1}{\gamma_n} \int_a^b f(x) \phi_n(x) w(x) dx$$

Caveat's

- Only easy to determine polynomial coefficients $\hat{f}_k$ analytically for a limited class of continuous functions.
- For arbitrary functions closed form expressions for coefficients are not known.

Similar to Fourier Methods we can try and resort to discrete approximations...

Discrete polynomials

We know how to determine the exact coefficients of the truncated polynomial expansion, but

- How can we determine unknown coefficients for a discrete polynomial expansion?
- Can we retain the fast rate of convergence (accuracy) in approximate methods?
- How can it be done efficiently?

Again, main problem is orthogonal projection that requires evaluation of integrals.

What can we do?

Discrete polynomial methods

Consider the coefficients

$$\hat{f}_n = \frac{1}{\gamma_n} \int_a^b f(x) \phi_n(x) w(x) dx$$

Introduce a discrete set of $N + 1$ nodes (**abscissas**) on the interval $x \in [a, b]$

$$a \leq x_0 < x_1 < \dots < x_N \leq b$$

which can be used to evaluate the integral via a **quadrature** rule of the form

$$\int_a^b f(x) \phi_n(x) w(x) dx \simeq \sum_{j=0}^N f(x_j) \phi_n(x_j) w_j$$

if we can determine a corresponding set of **weights** w_j .

How can we determine accurate quadrature rules of this form?

Numerical integration via Gauss-Type quadrature rules

For use with polynomial methods based on orthogonal polynomials we are interested in quadratures rules that satisfy

$$\int_a^b \phi_m(x)\phi_n(x)w(x)dx = \sum_{j=0}^N \phi_m(x_j)\phi_n(x_j)w_j$$

over the widest possible range of m and n for maximum accuracy.

The most accurate quadrature rules are the classical Gauss-Type quadrature rules of the form

$$\int_a^b f(x)w(x)dx \simeq \sum_{j=0}^N f(x_j)w_j$$

which corresponds to setting $f(x) = \phi_m(x)\phi_n(x)$ where $f(x) \in \mathbb{P}_{2N+\mu}$.

Gauss-Type quadrature rules

To derive a formula for approximating an integral as a sum

$$\int_a^b f(x)w(x)dx \simeq \sum_{i=0}^N f(x_i)w_i$$

using $N + 1$ quadrature points, we can invoke interpolation.

Assume, that $f(x) \in C^{N+1}([a, b])$ is represented in terms of an interpolating Lagrange polynomial of the form

$$\mathcal{I}_N f(x) = \sum_{i=0}^N f(x_i)h_i(x), \quad h_i(x) = \prod_{j=0, j \neq i}^N \frac{x - x_j}{x_i - x_j}$$

where the i 'th Lagrange polynomial is defined in terms of the unique set of abscissas x_j , $j = 0, 1, \dots, N$, chosen. The interpolation errors (remainder) is given as

$$\mathcal{R}_N f(x) = f(x) - \mathcal{I}_N f(x) = \frac{f^{(N+1)}(\xi)}{(N+1)!} q_N(x), \quad q_N(x) = \prod_{i=0}^N (x - x_i)$$

with $\xi \in [a, b]$.

Gauss-Type quadrature rules

A quadrature formula can be determined by the following method.

Do the weighted integration of $f(x) = \mathcal{I}_N f(x) + \mathcal{R}_N f(x)$ such that

$$\begin{aligned}\int_a^b f(x)w(x)dx &= \int_a^b \mathcal{I}_N f(x)w(x)dx + \int_a^b \mathcal{R}_N f(x)w(x)dx \\ &= \sum_{i=0}^N f(x_i)w_i + \frac{f^{(N+1)}(\xi)}{(N+1)!} \int_a^b q_N(x)w(x)dx\end{aligned}$$

where

$$w_i = \int_a^b h_i(x)w(x)dx, \quad i = 0, 1, \dots, N$$

Remark, this quadrature formula is exact if $f(x) \in \mathbb{P}_{N+1}$.

Can we do better?

Gauss-Type quadrature rules

Suppose that $f(x) \in \mathbb{P}_{2N+1}$ and can be decomposed as an interpolating polynomial of the form

$$f(x) = \left(\sum_{i=0}^N f(x_i) h_i(x) \right) + s(x) r(x)$$

where $h_i(x) \in \mathbb{P}_N$, $s(x) \in \mathbb{P}_N$ and $r(x) \in \mathbb{P}_{N-1}$.

Also, choose interpolation N nodes such that $s(x_i) = 0$.

How should we choose $s(x)$ and x_i optimally for maximum **accuracy** in quadrature rules?

Gauss-Type quadrature rules

By a weighted integration of $f(x)$ we find

$$\int_a^b f(x)w(x)dx = \left(\sum_{i=0}^N f(x_i) \int_a^b h_i(x)w(x)dx \right) + \int_a^b s(x)r(x)w(x)dx$$

or expressed in the more compact form

$$\int_a^b f(x)w(x)dx = \left(\sum_{i=0}^N f(x_i)w_j \right) + \int_a^b s(x)r(x)w(x)dx$$

where

$$w_j = \int_a^b h_i(x)w(x)dx, \quad \mathcal{R}f(x) = \int_a^b s(x)r(x)w(x)dx$$

Gauss-Type quadrature rules

If we restrict our interest to integration over intervals $x \in [-1, 1]$ with respect to the weight function for Jacobi polynomials

$$w(x) = (1-x)^\alpha(1+x)^\beta$$

Then we find

$$w_j^{(\alpha,\beta)} = \int_{-1}^1 (1-x)^\alpha(1+x)^\beta h_j(x) dx$$
$$\mathcal{R}f(x) = \int_{-1}^1 (1-x)^\alpha(1+x)^\beta s(x)r(x) dx$$

If we choose $s(x) = P_N^{(\alpha,\beta)}$ and $r(x) \in \mathbb{P}_{N-1}$ then

$$\mathcal{R}f(x) = \int_{-1}^1 (1-x)^\alpha(1+x)^\beta P_N^{(\alpha,\beta)}(x)r(x) dx = 0$$

due to the orthogonal properties of the Jacobi polynomials. By determining the N zeroes x_i of $P_N^{(\alpha,\beta)}(x)$ we find the classical Gauss quadrature rule.

Inner product notations

We assume intervals $x \in [-1, 1]$ for defining notations.

Continuos case

$$(u, v)_w = \int_{-1}^1 u(x)v(x)w(x)dx$$
$$||v||_w = \sqrt{(v, v)_w}$$

Discrete case

$$(u, v)_N = \sum_{j=0}^N u(x_j)v(x_j)w_j$$
$$||v||_N = \sqrt{(v, v)_N}$$

In the discrete case, high-order accurate gaussian quadrature are typically used for approximation of continuous integrals.

Accuracy of Gauss-Type quadrature rules

The accurate Gauss-Type integration formulas imply that the orthogonality properties are preserved for weighted inner products of the form

$$(u, v)_N = (u, v)_w \quad \text{if } uv \in \mathbb{P}_{2N+\mu}$$

and only requires $N + 1$ interpolation nodes to integrate polynomials of up to order $2N + \mu$ exactly.

In summary, depending on choice of abscissas the resulting rules will have different accuracy and are named according to

μ	Type	Interval
1	Gauss (no end points)	$] -1, 1[$
0	Gauss-Radau (one end point)	$[-1, 1[$
-1	Gauss-Lobatto (two end points)	$[-1, 1]$

How to determine quadrature nodes x_i and weights w_i ?

Chebyshev Gauss Quadrature

Gauss Quadrature based on Chebyshev polynomials are widely used since fast evaluation can be done using the FFT transform. The weight function is $w(x) = (\sqrt{1-x^2})^{-1}$.

This reduces the Jacobi Gauss Quadrature to

$$x_j = \cos\left(\frac{2j+1}{2N+2}\pi\right), \quad j = 0, 1, \dots, N$$
$$w_j = \frac{\pi}{N+1}$$

This quadrature rule is exact for all $f(x) \in \mathbb{P}_{2N+1}$.

To compute the abscissas and weights we can

- Compute the roots $x_j \in]-1, 1[$ directly from the given explicit relation, and
- Evaluate the quadrature weights using the explicit formula.

This is the only quadrature rule which both abscissas and weights given explicitly.

Abscissas and weights for Gauss-Type quadrature

For flexible-order Gauss-Type quadrature rules we need nodes and corresponding weights.

For finding the abscissas we can

- Compute abscissas from explicit formulas (only Chebyshev).
- ~~Resort to tabulated abscissas.~~
- Compute abscissas one by one using some iterative strategy.
- Compute abscissas from the solution of an eigenvalue problem.

For finding weights we can

- ~~Resort to tabulated abscissas.~~
- Compute weights from explicit formulas (possible with abscissas given).
- Compute weights from the solution of an eigenvalue problem.

Zeroes of Jacobi polynomials

Few explicit closed form formulas for roots of orthogonal polynomials exist, e.g. Jacobi Polynomials $\phi_N(x) = P_N^{(\alpha,\beta)}(x)$.

Fortunately, the roots can be computed **iteratively** one by one using the Newton-Raphson method

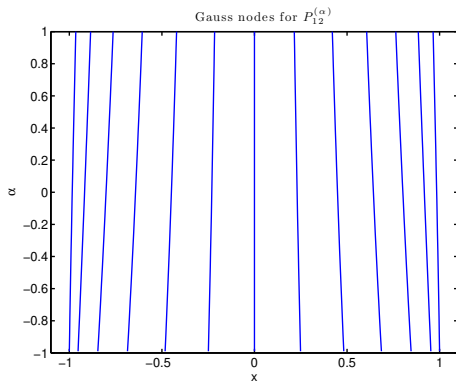
$$x_j^{k+1} = x_j^k + \delta^k, \quad \delta^k = -\frac{\phi_N(x_j^k)}{\phi'_N(x_j^k)}, \quad k = 0, 1, \dots, k_{\max}$$

For fast convergence, **initial guess** is typically defined using the explicit formula for Chebyshev abscissas or other formulas for asymptotic representations of roots.

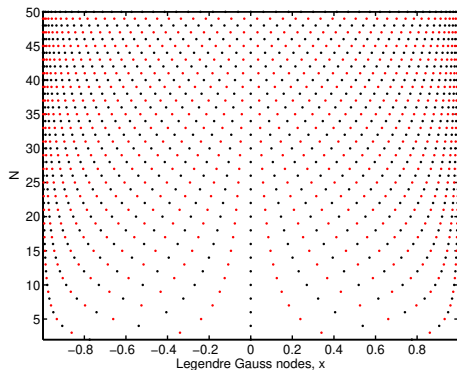
The iterations is stopped when $|\delta| < \text{TOL}$ with tolerance typically a small factor of machine precision (fx. $\text{TOL} = 5\epsilon$).

Zeroes of Jacobi polynomials

Illustration of the **Gauss nodes** $x_i \in]-1, 1[$ of Jacobi polynomials.



(a) $P_{12}^{(\alpha)}$ (Ultraspherical).



(b) $P_N^{(0,0)}$ (Legendre)

Calculation of Gauss Quadrature Rules

Golub and Welsch (1969) showed how Gauss quadrature rules can be generated from a symmetric tridiagonal matrix generated from the three term recurrence for any set of orthogonal polynomials $\{p_n(x)\}_{n=0}^N$ associated with a weight function $w(x)$ on $x \in [a, b]$.

Any such set of polynomials satisfies a three term recurrence of the form

$$p_{-1}(x) = 0,$$

$$p_0(x) = 1$$

$$p_n(x) = (a_n x + b_n)p_{n-1}(x) - c_n p_{n-2}(x), \quad n = 1, 2, \dots, N$$

with $a_n > 0$, $c_j > 0$.

Calculation of Gauss Quadrature Rules

We can express the first N polynomials as a coupled system

$$x \begin{bmatrix} p_0(x) \\ p_1(x) \\ \vdots \\ p_{N-1}(x) \end{bmatrix} = \begin{bmatrix} -\frac{b_1}{a_1} & \frac{1}{a_1} & & \\ \frac{c_2}{a_2} & -\frac{b_2}{a_2} & \frac{1}{a_2} & \\ & \ddots & \ddots & \\ & & \frac{c_N}{a_N} & -\frac{b_N}{a_N} \end{bmatrix} \begin{bmatrix} p_0(x) \\ p_1(x) \\ \vdots \\ p_{N-1}(x) \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ \vdots \\ \frac{p_N(x)}{a_N} \end{bmatrix}$$

or equivalently in compact matrix notation

$$x\mathbf{p}(x) = T\mathbf{p}(x) + \frac{1}{a_N}p_N\mathbf{e}_N$$

with T a tridiagonal matrix and $\mathbf{e}_N = (0, 0, \dots, 1)^T$ the N 'th unit vector.

Any root x_n of the polynomial $p_N(x_n)$, $n = 1, \dots, N$ should satisfy

$$x_n\mathbf{p}(x_n) = T\mathbf{p}(x_n)$$

which shows that x_n , $n = 1, 2, \dots, N$, are eigenvalues of T .

Calculation of Gauss Quadrature Rules

If T is not symmetric it is possible to normalize the polynomials by performing a diagonal similarity transformation

$$DTD^{-1} = J = \begin{bmatrix} \alpha_1 & \beta_1 & 0 & & & \\ \beta_1 & \alpha_2 & \beta_2 & & & \\ & \ddots & \ddots & \ddots & & \\ & & \ddots & \ddots & \beta_{N-1} & \\ & & & \beta_{N-1} & \alpha_N & \end{bmatrix}$$

where

$$\alpha_n = -\frac{b_n}{a_n}, \quad \beta_n = \left(\frac{c_{n+1}}{a_n a_{n+1}} \right)^{1/2}$$

If the eigenvectors of T are calculated in terms of normalized eigenvectors

$$J\mathbf{q}_n = x_n \mathbf{q}_n, \quad n = 1, 2, \dots, N$$

with discrete inner product $\mathbf{q}_n^T \mathbf{q}_n = 1$.

Calculation of Gauss Quadrature Rules

Then by the Christoffel-Darboux identity it can be shown that the first element of the sets of eigenvectors are related such that

$$q_{1,n}^2 = w_n(p_0(x_n))^2, \quad n = 1, 2, \dots, N$$

from which we find the following formula for computing Gauss quadrature weights

$$w_n = \frac{q_{1,n}^2}{p_0^2(x_n)}, \quad n = 1, 2, \dots, N$$

Jacobi Gauss quadrature

Integrals can be determined using high-order accurate **Gaussian quadrature rules** of the form

$$\int_{-1}^1 u(x)(1-x)^\alpha(1+x)^\beta dx = \sum_{n=0}^N u(x_n)w_n + \mathcal{R}(u)$$

which can exactly integrate polynomials $u(\xi) \in \mathbb{P}_{2N+2-k}$ if it is based on **Gauss points** ($k = 1$), e.g. the Gauss-Legendre quadrature ($\alpha = \beta = 0$). In Matlab compute Jacobi Gauss rule for arbitrary N

```
>> [x,w]=JacobiGQ(alpha,beta,N);
```

suitable for determining weighted integrals of the form given.

Remark that typically the weight function contribution $w(x) = (1-x)^\alpha(1+x)^\beta$ is absorbed into quadrature weights w_n .

Legendre Gauss Quadrature

Gauss Quadrature based on Legendre polynomials $L_n(x) = P_n^{(0,0)}(x)$ are widely used since $w(x) = 1$.

This reduces the Legendre Gauss quadrature rule to

$$x_j = \text{zeroes of } L_{N+1}(x)$$
$$w_j = \frac{2}{(1 - x_j^2)(L'_{N+1}(x_j))^2}$$

This quadrature rule is exact for all $f(x) \in \mathbb{P}_{2N+1}$.

To compute the abscissas and weights we must

- Find the roots $x_j \in]-1, 1[$ of the $N + 1$ order Legendre polynomial $L_{N+1}(x)$, and
- Evaluate the quadrature weights using formula (requires knowledge of abscissas for evaluation of Lagrange polynomial).

Legendre Gauss-Lobatto Quadrature

Gauss quadrature rules which include the endpoints $x = \pm 1$ are referred to as Gauss-Lobatto quadrature.

The abscissas and weights can in this case be determined from

$$x_j = \pm 1, \text{ zeroes of } L'_N(x), \quad j = 0, 1, \dots, N$$
$$w_j = \frac{2}{N(N+1)} \frac{1}{L_N(x_j)^2}$$

Since two abscissas has been determined, there are 2 degrees of freedom less. Thus, this quadrature rule is exact for all $f(x) \in \mathbb{P}_{2N-1}$.

To compute the abscissas and weights we can

- Find the roots $x_j \in]-1, 1[$ of the $N - 1$ order polynomial $L'_N(x)$, and
- Evaluate the quadrature weights using the explicit formula.

Chebyshev Gauss-Lobatto Quadrature

The abscissas and weights can in this case be determined from

$$x_j = \cos\left(\frac{j\pi}{N}\right), \quad j = 0, 1, \dots, N$$
$$w_j = \begin{cases} \frac{\pi}{2N} & , j = 0, N \\ \frac{\pi}{N} & , j = 1, 2, \dots, N-1 \end{cases}$$

This quadrature rule is exact for all $f(x) \in \mathbb{P}_{2N-1}$.

To compute the abscissas and weights we can

- Compute the roots $x_j \in [-1, 1]$ directly from the given explicit relation, and
- Evaluate the quadrature weights using the explicit formula.

Discrete polynomial transform

Consider the coefficients

$$\hat{f}_k = \frac{1}{\gamma_n} \int_{-1}^1 f(x) \phi_k(x) w(x) dx$$

Introduce a discrete set of $N + 1$ nodes on the interval $x \in [-1, 1]$ such that the nodes corresponds to the abscissas of the high-order accurate Gauss-Type quadrature rules.

Then a discrete approximation of the polynomial coefficients is given as

$$\tilde{f}_k = \frac{1}{\gamma_k} \sum_{j=0}^N f(x_j) \phi_k(x_j) w_j, \quad \gamma_k = \sum_{j=0}^N \phi_k(x_j)^2 w_j$$

How fast convergence do we get?

Discrete polynomial transform

If we use the discrete coefficients with an expansion for polynomials with $N + 1$ unknown coefficients

$$f(x) \approx \sum_{k=0}^N \tilde{f}_k \phi_k(x)$$

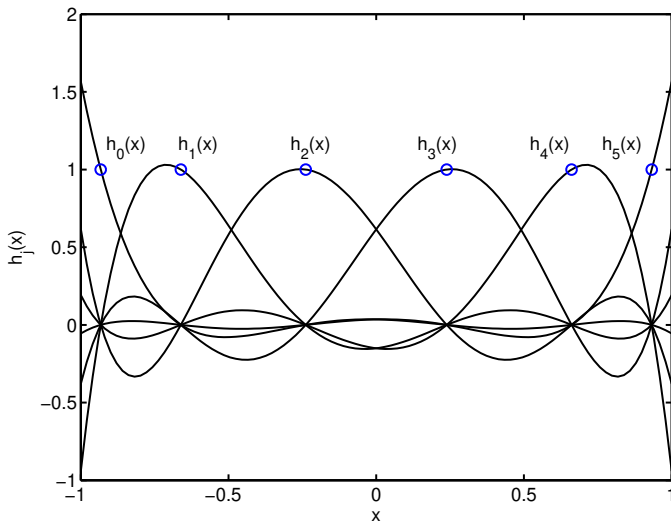
Insert the expression for the discrete coefficients and we find

$$f(x) \approx \sum_{k=0}^N \left(\frac{1}{\gamma_k} \sum_{j=0}^N f(x_j) \phi_k(x_j) w_j \right) \phi_k(x)$$

If we change the order of summation we find

$$f(x) \approx \sum_{j=0}^N f(x_j) h_j(x), \quad h_j(x) = \sum_{k=0}^N \frac{1}{\gamma_k} \phi_k(x_j) \phi_k(x) w_j$$

Polynomial interpolation



(c) Lagrange polynomials based on 6 Legendre-Gauss abscissas.

Polynomial interpolation

We find that $h_j(x)$ is a Lagrange polynomial for given abscissas

$$h_j(x_i) = w_j \sum_{k=0}^N \frac{1}{\gamma_k} \phi_k(x_j) \phi_k(x_i) = \delta_{ij}$$

This result implies that the discrete approximation of the function $f(x)$ is in fact the **polynomial interpolant** satisfying

$$\mathcal{I}_N f(x_i) = f(x_i), \quad i = 0, 1, \dots, N$$

Polynomial interpolation

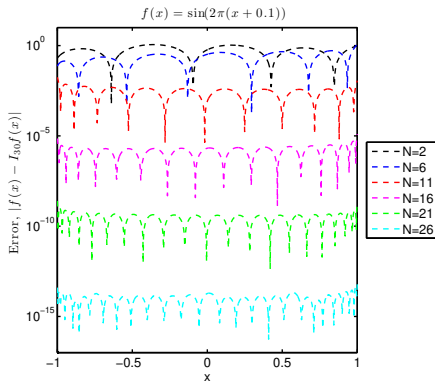
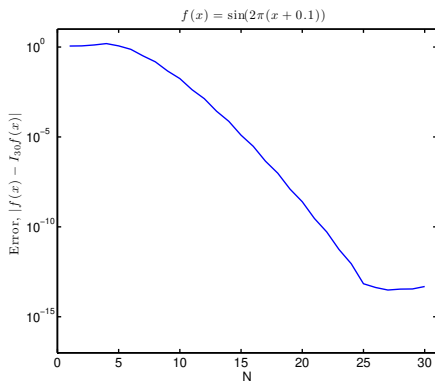
Consider the approximation of the $C^\infty([-1, 1])$ function

$$f(x) = \sin(2\pi(x + 0.1))$$

by a polynomial expansion based on discrete expansion coefficients

$$\tilde{f}_k = \frac{1}{\gamma_k^{(0,0)}} \sum_{j=0}^N f(x_j) P_k^{(0,0)}(x_j) w_j$$

determined via Legendre-Gauss quadrature rules.



Discrete polynomial transform

The Discrete Polynomial Transform (DPT) is the mapping between the discrete set of function values $f(x_j)$ and the polynomial coefficients $\hat{f}_k$ of the polynomial expansion.

In summary, the $(N + 1)$ -point DPT is defined as

$$\hat{f}_k = \frac{1}{\gamma_k} \sum_{j=0}^N f(x_j) \phi_k(x_j) w_j, \quad \gamma_k = \sum_{j=0}^N \phi_k(x_j)^2 w_j$$

and the inverse DPT (IDPT) as

$$f(x_j) = \sum_{k=0}^N \hat{f}_k \phi_k(x_j)$$

The DPT operations can be computed efficiently via recurrence relations for polynomials. and in the case of Chebyshev polynomials also via the efficient FFT method due to Cooley and Turkey (1967).

Discrete polynomial transform in practice

The interpolating polynomial of a function $f(x)$ is represented as

$$\mathcal{I}_N f(x) = \sum_{k=0}^N \tilde{f}_k \phi_k(x)$$

and satisfies

$$\mathcal{I}_N f(x_j) = f(x_j), \quad j = 0, 1, \dots, N$$

The discrete coefficients $\tilde{f}_k$ is found via the discrete inner product

$$\tilde{f}_k = \frac{1}{\tilde{\gamma}_n^{(\alpha, \beta)}} \sum_{j=0}^N f(x_j) P_n^{(\alpha, \beta)}(x_j) w_j$$

which for [practical computations](#) are often computed in the form

$$\tilde{f}_k = \sum_{j=0}^N f(x_j) \tilde{P}_n^{(\alpha, \beta)}(x_j) w_j$$

The weights w_j are dependent on the choice of abscissas x_j .

Modal or nodal representation

The duality between using a model or nodal interpolating polynomial representation is related through the choice of modal basis functions $\phi_n(x)$ and the distinct nodal interpolation points $x_i \in [-1, 1]$, $i = 0, 1, \dots, N$.

We can express this as

$$f(x_i) = \sum_{n=0}^N \hat{f}_n \phi_n(x_i) = \sum_{n=0}^N f_n h_n(x_i), \quad i = 0, 1, \dots, N$$

which defines a relationship between modal and nodal coefficients

$$\mathbf{f} = \mathcal{V} \hat{\mathbf{f}}$$

where $\mathcal{V}$ is a generalized Vandermonde matrix and $\mathbf{f}$ nodal and $\hat{\mathbf{f}}$ modal coefficients defined as

$$\mathcal{V}_{ij} = \phi_j(x_i), \quad \mathbf{f} = (f_0, \dots, f_N)^T, \quad \hat{\mathbf{f}} = (\hat{f}_0, \dots, \hat{f}_N)^T$$

Discrete operators

With a relationship between the coefficients of our modal and nodal expansions

$$\mathbf{f} = \mathcal{V} \hat{\mathbf{f}}$$

the modal and nodal bases can be related through

$$\mathbf{f}^T \mathbf{h}(x) = \hat{\mathbf{f}}^T \phi(x)$$

These relationships can be combined in order to connect basis functions through $\mathcal{V}$

$$\mathbf{f}^T \mathbf{h}(x) = \hat{\mathbf{f}}^T \phi(x)$$

$$\Rightarrow \left(\mathcal{V} \hat{\mathbf{f}} \right)^T \mathbf{h}(x) = \hat{\mathbf{f}}^T \phi(x)$$

$$\Rightarrow \hat{\mathbf{f}}^T \mathcal{V}^T \mathbf{h}(x) = \hat{\mathbf{f}}^T \phi(x)$$

$$\Rightarrow \mathcal{V}^T \mathbf{h}(x) = \phi(x)$$

Discrete operators

The compact relationship

$$\mathcal{V}^T \mathbf{h}(x) = \phi(x)$$

can be expressed in the form

$$\begin{bmatrix} \phi_0(x_0) & \phi_0(x_1) & \cdots & \phi_0(x_N) \\ \phi_1(x_0) & \phi_1(x_1) & \cdots & \phi_1(x_N) \\ \vdots & \vdots & \ddots & \vdots \\ \phi_N(x_0) & \phi_N(x_1) & \cdots & \phi_N(x_N) \end{bmatrix} \begin{bmatrix} h_0(x) \\ h_1(x) \\ \vdots \\ h_N(x) \end{bmatrix} = \begin{bmatrix} \phi_0(x) \\ \phi_1(x) \\ \vdots \\ \phi_N(x) \end{bmatrix}$$

It turns out that we can exploit this connection for defining useful discrete operations.

Discrete operators

The i 'th Lagrange polynomial can be expressed as

$$h_i(x) = \sum_{n=0}^N (\mathbf{v}^T)_{in}^{-1} \phi_n(x)$$

Consider the L^2 -inner product

$$\|f\|_{L_2}^2 = \int_{-1}^1 f(x)^2 dx$$

Represent $f(x)$ using a an interpolating nodal polynomial approximation of the form

$$f(x) = \sum_{j=0}^N f_j h_j(x) = \mathbf{f}^T \mathbf{h}(x)$$

having chosen suitable abscissas.

Discrete operators

Then insert expansion into L^2 -inner product

$$\|f\|_N^2 = \int_{-1}^1 (\mathbf{f}^T \mathbf{h}(x)) (\mathbf{f}^T \mathbf{h}(x)) dx$$

and rewrite in terms of modal basis functions such that

$$\begin{aligned} \|f\|_N^2 &= \int_{-1}^1 (\mathbf{f}^T \mathbf{h}(x)) (\mathbf{h}(x)^T \mathbf{f}) dx \\ &= \mathbf{f}^T \left(\int_{-1}^1 (\mathcal{V}^T)^{-1} \tilde{\phi}(x) \tilde{\phi}(x)^T (\mathcal{V})^{-1} dx \right) \mathbf{f} \\ &= \mathbf{f}^T (\mathcal{V} \mathcal{V}^T)^{-1} \mathbf{f} \end{aligned}$$

having assumed orthonormality of the basis functions.

Thus, the discrete inner product can be evaluated through a [matrix-based integration](#) on chosen abscissas on $x \in [-1, 1]$

$$\|f\|_N^2 = \mathbf{f}^T \mathcal{M} \mathbf{f}, \quad \mathcal{M} = (\mathcal{V} \mathcal{V}^T)^{-1}$$

Discrete operators

Computing the first derivative approximation in nodal (physical space) can be done by a matrix-vector product of the form

$$\frac{d}{dx} \left(\sum_{j=0}^N f_j h_j(x) \right) = \sum_{j=0}^N f_j \frac{d}{dx} h_j(x) \quad \rightarrow \quad \frac{d\mathbf{f}}{dx} = \mathcal{D}\mathbf{f}, \quad \mathcal{D}_{ij} = \left. \frac{dh_j}{dx} \right|_{x=x_i}$$

on the chosen grid. How can we determine $\mathcal{D}$?

Using a modal expansion we can express the first derivative as

$$\frac{d}{dx} \left(\sum_{j=0}^N \hat{f}_j \phi_j(x) \right) = \sum_{j=0}^N \hat{f}_j \frac{d}{dx} \phi_j(x) \quad \rightarrow \quad \frac{d\mathbf{f}}{dx} = \mathcal{V}_x \hat{\mathbf{f}}, \quad \mathcal{V}_{x,ij} = \left. \frac{d\phi_j}{dx} \right|_{x=x_i}$$

By combining and relating these operation to modal coefficients we find the **differentiation matrix** for use on $x \in [-1, 1]$

$$\frac{d\mathbf{f}}{dx} = \mathcal{D}\mathbf{f} = \mathcal{D}\mathcal{V}\hat{\mathbf{f}} = \mathcal{V}_x \hat{\mathbf{f}} \quad \rightarrow \quad \mathcal{D} = \mathcal{V}_x \mathcal{V}^{-1}$$

The elements of the discrete operators can be determined using the standard three-term recurrences.

Discrete operators and alternatives

Discrete matrix-based operators can also be constructed for

- Interpolation
- High-order differentiation

These operators has a work complexity of $\mathcal{O}(N^2)$ (**bad scaling**).

Some alternative strategies to matrix-based operations are

- Directly evaluate derivatives via formulas at $\mathcal{O}(n)$ cost.
(e.g. via expression for derivatives of Lagrange polynomials)
- Transform methods at $\mathcal{O}(n)$ for evaluating derivatives or $\mathcal{O}(N \log(N))$ costs.
(e.g. FFT for Fourier and Chebyshev transforms if possible).

Choice between methods is typically based on **accuracy** (accumulation of rounding errors with N) and **efficiency** concerns.